

## Lab 3: Architectural Pattern Selection – Justification

**NAME: K M PRADHYUT**

**SRN : PES1UG24CS646**

**SEC: K**

**Architecture Selection:** We chose the Microservices Architecture for the Second-Hand Product Escrow Marketplace System.

**Architectural Choice:** Microservices. The system is split into eight independently deployable components behind one API Gateway: User & Auth, Listing, Order, Inspection, Dispute, Escrow & Payment and Notification Services, plus the external Payment Gateway. Each service owns its datastore and exposes a provided interface (IAuth, IListing, IOrder, IInspection, IDispute, IReport, INotify, IEscrow, IFreeze, IPayment) using REST, gRPC/TLS, or an asynchronous event queue. Layered architecture was rejected because one deployable unit would couple browsing traffic to money-handling code, so a slow payment call or a listing bug could stall escrow, and no part could be scaled or isolated on its own. Client-Server was rejected because a single central server is a single point of failure and a bottleneck, which is unacceptable when funds must be frozen or released reliably (FR-001, FR-002, NFR-001).

**Reason 1 – Fault isolation around escrow:** The marketplace depends on an external Payment Gateway that can time out or reject transfers (UC-06, exception flow E1). In our design this failure stays inside the Escrow & Payment Service: the order remains in 'Approved', funds stay locked, the service retries three times and then flags the order for Admin review, while listing, inspection, dispute and notification keep working. Likewise, a dispute takes precedence over a pending approval because the Dispute Service calls IFreeze on the Escrow Service directly, so funds are frozen the moment a Buyer raises a dispute, without waiting for other components.

**Reason 2 – The domain splits naturally into services with different change and scale profiles:** Our use cases map almost one-to-one onto services (UC-01 Auth, UC-02 Listing, UC-03/UC-06 Order, UC-04/UC-08 Escrow, UC-05 Inspection, UC-07/UC-09 Dispute, FR-005 Notification). Listing and browsing are read-heavy, while escrow is low-volume but must be strictly correct. The Order Service is the single owner of the order state machine (Delivered → Approved → Completed), so funds can be released only from 'Approved', as the business rules require. Because components are replaceable behind their interfaces, the payment provider can be swapped by changing only the IPayment integration.

**Security Advantage:** Sensitive financial data is confined to one service. Only the Escrow & Payment Service talks to the Payment Gateway and its own encrypted datastore, with TLS enforced on every inter-service call and AES-256 at rest (NFR-002), so a compromise of Listing or Notification cannot expose card or escrow data. The API Gateway is the only public entry point: it validates tokens through IAuth and enforces role-based access, so that only the Buyer on an order can approve delivery or raise a dispute and only the assigned Inspection Agent can submit a report. IFreeze also lets the Dispute Service block any withdrawal immediately while an investigation is open (NFR-001).

**Performance Benefit:** Each service scales independently: extra Listing instances (with caching) absorb browsing peaks without adding cost to the Escrow Service. Status notifications travel over an asynchronous event queue, so an order-state change never blocks on email or in-app delivery, and alerts can reach both parties within the one-minute target (FR-005). The dispute freeze is a lightweight synchronous gRPC call to a dedicated service, so it can complete within the five-second target (FR-002) even under peak load.